

help me decide for captain jays website the tech stack and templates etc the goal is to rework captainjays.net website into a professional, smooth, fast and good looking websites like popeyes, davs hot chicken, chipotle, canes etc inshallah really good, also captain jays website has two different types of restaurants ebt and non ebt so when a customer comes on the website the homepage and menu page loads based on the option they select ebt or fried store, the website should have a blog page and a management system for blogs for seo and advertisement i am currently looking for a really good template that has THIS and thinking whether to go with nextjs or transtack

Web search do in-depth search me act as senior full stack developer

Worked for 1m 34s

Yes. I looked at Captain Jay's current site, the restaurant sites you want to compete with, the current state of Next.js/TanStack Start, and CMS/template options. I would **not** treat this as a normal restaurant website redesign. Captain Jay's has enough locations, two distinct store experiences, online ordering, promotions, SEO needs, and ongoing content management that this should be built as a small restaurant web platform.

My recommendation

For Captain Jay's, I would build:

Next.js 16 App Router + TypeScript + Tailwind CSS v4 + shadcn/ui + Sanity CMS + Vercel

Use **Motion/Framer Motion selectively** for animations, and build the actual Captain Jay's visual identity custom rather than shipping a generic restaurant theme.

I would **not choose TanStack Start for this project right now.**

TanStack Start is genuinely good: SSR, streaming, type-safe routing, server functions, Vite, multiple deployment targets, and solid SEO capabilities. But as of today it is still officially **Release Candidate**, whereas Next.js has a far larger production ecosystem, mature CMS integrations, restaurant/CMS starter templates, built-in metadata/sitemap/robots/OG handling, image/font optimizations, and official Sanity tooling. 🌟 TanSt... +2

For a Captain Jay's production website where the company wants you to own this long-term, I'd choose boring reliability over framework novelty.

Framework decision

	Next.js 16	TanStack Start
Production maturity	Excellent	RC
CMS ecosystem	Excellent	Decent
Sanity integration	Official / excellent	Possible but more manual
SEO tooling	Excellent	Excellent
Restaurant templates	Many	Very few
Vercel deployment	First-class	Supported
Server rendering	Yes	Yes
Static generation	Yes	Yes
Dynamic routes	Yes	Yes
Developer experience	Excellent	Excellent
Type-safe routing	Good	Outstanding
Risk for Captain Jay's	Low	Medium
My choice	● YES	● Not for this project yet

TanStack Start's architecture is attractive, especially if this were a greenfield developer-focused SaaS. Its router is arguably nicer than Next routing for complex applications. But this website is fundamentally a **content, local SEO, restaurant marketing and conversion platform**. Next.js has the advantage where you actually need it.

The bigger architectural decision: don't make EBT/Fried just a React toggle

This matters a lot.

You said:

customer comes to website → chooses EBT or Fried store → homepage/menu loads accordingly.

That's a good UX concept, but I would **not** implement:

captainjays.net → React state says isEBT=true → completely different site rendered at the exact same URL.

It makes analytics, sharing, ads, SEO and indexing needlessly complicated.

Google explicitly recommends that individual pieces of content in JS applications have their own crawlable URLs, and server/pre-rendering remains recommended for users and crawlers.  Google Develo... +1

Instead I'd architect it like this:

```
captainjays.net/
├── first visit / no preference
│   └── ↓
│       "What are you looking for?"
│       ┌──  Fried Restaurant ─┐
│       └──  Fresh Market/EBT ─┘
├── /fried
│   ├── /fried/menu
│   └── fried-focused marketing
├── /market
│   ├── /market/menu
│   └── EBT/market-focused marketing
├── /locations
│   ├── /locations/dearborn-mi
│   ├── /locations/detroit-7-mile
│   └── /locations/inkster-mi
├── /blog
│   └── /blog/[slug]
├── /about
├── /careers
└── /contact
```

Save their preference in a cookie such as:

```
cj_concept=fried
```

or

cj_concept=market

On future visits, you can take them directly to their preferred experience while always keeping a highly visible **"Switch to Fresh Market / Switch to Fried"** control.

That gives you the personalization you want while preserving real, crawlable URLs.

Even better: store selection should eventually be the real personalization

The concept choice gets them into the correct Captain Jay's world.

Afterward, the user should select a location.

Then your application knows:

TypeScript

```
location = {
  name: "Captain Jay's ...",
  slug: "...",
  concept: "market", // or fried
  acceptsEBT: true,
  onlineOrderingEnabled: true,
  orderUrl: "...",
  address: "...",
  phone: "...",
  hours: [...],
  latitude: ...,
  longitude: ...
}
```

Now everything gets smarter.

The header can say:

Your Store: Dearborn — Change

And:

- Order Now goes to the proper ordering location.
- Hours become accurate.
- Menu can reflect applicable availability.
- EBT messaging is correct.
- Promotions can be store-specific.
- Location SEO becomes excellent.

Dave's Hot Chicken is already doing something important here. Individual restaurant locations have standalone indexable pages containing address, phone, hours, amenities, local content, menu information and ordering CTAs.  Dave's Hot Chi...

Captain Jay's should do the same.

Google specifically supports `Restaurant` / `LocalBusiness` structured data with things such as location, hours, phone and menu URL.  Google Develo... +1

With 20+ Captain Jay's locations, this could become one of the most valuable SEO improvements you make.

The CMS is where I think Sanity wins

You do **not** want somebody calling you six months from now saying:

| Ahthesham, change the homepage banner.

| Ahthesham, add this Ramadan/Eid promotion.

| Change this store hour.

| Publish this article.

| Make this ad active tomorrow.

| Add this menu photo.

That is exactly why I would separate:

code/layout/design → Next.js

from

content/menu marketing/blog/promotions → Sanity.

Sanity has a very good official Next.js integration including App Router support, Draft Mode, live content, visual editing and click-to-edit previews. Editors can see the actual webpage and click the element they want to modify.  Sanit... +1

It also now supports scheduling drafts for future publishing, which is useful for campaigns and blog content.  Sanit...

For example, a manager could prepare:

"Super Bowl Weekend — 50 Wings Special"

Wednesday, configure it to start Friday morning, then have it disappear automatically Sunday night.

No deployment.

No developer.

This is the Sanity content model I'd build

This is much more important than which template you buy.

```
Sanity Studio
|
|— Site Settings
|   |— Logo
|   |— Social links
|   |— Header
|   |— Footer
|   |— Global SEO
|
|— Concepts
|   |— Fried Restaurant
|   |— Fresh Market / EBT
|
|— Locations
|   |— Name
|   |— Address
|   |— Phone
|   |— Hours
|   |— Lat/Lng
|   |— Concept
|   |— EBT accepted
|   |— Ordering URL
|   |— Amenities
|   |— Store images
|
|— Menu
|   |— Categories
|   |— Items
|   |— Images
|   |— Description
|   |— Dietary information
|   |— Concept applicability
|
|— Promotions / Campaigns
|   |— Title
|   |— Creative
|   |— CTA
|   |— Fried / Market / Both
|   |— Applicable locations
|   |— Start date
|   |— End date
```

```
| | | — Homepage placement
| | | — Menu placement
| | | — Tracking campaign ID
|
| — Blog
| | | — Title
| | | — Slug
| | | — Author
| | | — Categories
| | | — Content
| | | — Featured image
| | | — Publish date
| | | — Related menu items
| | | — Related locations
| | | — SEO metadata
|
| — Pages
| | — Modular page blocks
|
| — Alerts
| | — Store closure
| | — Holiday hours
| | — Global announcement
```

This lets Captain Jay's marketing team effectively operate the website without touching GitHub.

Very important: don't make Sanity the POS

There's one boundary I'd enforce strongly.

If Paytronix/NCR is the actual ordering system and source of truth for pricing and availability, **don't manually maintain transactional prices in both Paytronix and Sanity.**

Otherwise eventually:

Website: 10 wings \$12.99

Paytronix: 10 wings \$14.99

Store POS: \$14.99

and you'll be the person everybody calls. 😊

Use Sanity for marketing information, photography, descriptions, SEO, category presentation and promotions.

Let the transactional system remain authoritative for actual ordering/pricing whenever possible.

The website's **Order Now** action should hand the customer into the appropriate Paytronix ordering flow.

Your current site already sends **ORDER NOW** to a separate Captain Jay's ordering experience, so maintaining that separation is consistent with the present architecture.

 Captain Jay...

The current Captain Jay's menu is one of the first things I'd kill

When I inspected the current site, Google essentially sees the menu pages as:

Swipe Left Or Right
Click to Open Full View

for both Fried and Fresh Market.  Captain Jay's ... +1

That's a huge missed opportunity.

Menu content should be **actual HTML**, not just images.

Instead Google should be able to understand:

Chicken

2 Piece Chicken
3 Piece Chicken
4 Piece Chicken
Chicken Sandwich
Chicken Tenders

Fish

Catfish
Whiting
Tilapia

Sides

Fries
Mac & Cheese
Okra
...

Dave's current menu, for comparison, exposes menu items, names, descriptions, categories and prices as real content that search engines can understand.  Dave's Hot Chi...

That is what Captain Jay's should move toward.

Template research: here's where it gets interesting

I would not search for one restaurant template and force the entire Captain Jay's architecture into it.

There are actually two separate things you need:

a strong technical starter

and

a strong visual direction.

The best technical starters I found are much better than most restaurant templates.

Starter/template	What I think
SanityPress	🏆 Best starting architecture
SiteEngine	🏆 Excellent clean technical foundation
Grillhouse	🏆 Closest all-in-one restaurant template
Maison	Excellent visual restaurant reference
Brasa/Webflow	Great fast-food design inspiration
Generic ThemeForest restaurant themes	❌ Would avoid

SanityPress — my favorite base

SanityPress already has:

Next.js, Tailwind 4, Sanity, visual editing, modular page builder, blog, redirects and reusable content modules. Its current project also advertises very strong Lighthouse performance. 

SanityPress project

This is extremely close to the **architecture** I would have built myself.

I would throw away most of its stock visual identity and create Captain Jay's UI on top of its CMS/page-builder structure.

That saves a lot of boring engineering.

SiteEngine is another very good option

It already combines:

Next.js + Sanity + TypeScript + Tailwind + shadcn/ui + Vercel

and has a page builder, Visual Editing, Live Preview and a form builder.  Sanity...

SiteEngine template

Technically this may actually be the cleanest match to the stack I'm recommending.

I would inspect both SanityPress and SiteEngine before writing any CMS boilerplate yourself.

Grillhouse is the closest "one template does everything"

This one caught my attention because it already includes:

Next.js + Sanity + menu + blog + FAQ + site settings + animations.  Sanity...

Grillhouse template

If you said:

Ahthesham needs something working ASAP; which existing restaurant template gets closest?

I'd say **Grillhouse**.

But there is one reason I wouldn't make it my first choice:

It uses **Chakra UI v2 + GSAP**.

For Captain Jay's I would prefer:

Tailwind v4 + shadcn/ui + custom components

because you will have far more design control and it'll be easier to create that Popeyes/Dave's/Chipotle style instead of inheriting a restaurant-theme feel.

Maison looks better visually

Maison is Next.js + Sanity and has editable menus, gallery, animated sections, featured dishes and a proper menu page.  Sanity...

Maison restaurant template

But its aesthetic is much more:

nice restaurant / café

than:

large aggressive fast-food brand.

Captain Jay's needs much more personality.

I actually like Brasa for Captain Jay's visual direction

This is a Webflow template, so I **wouldn't use Webflow for the actual Captain Jay's application**, but visually it is much closer to the space you want.

It has bold typography, red tones, high-contrast fast-food branding, animation, and importantly, the listing says a Figma file is included. 

So one possible workflow is:

```
SanityPress / SiteEngine
+
Captain Jay's custom CMS schema
+
visual direction inspired by Brasa / Dave's / Popeyes
↓
custom Next.js implementation
```

That is a much stronger strategy than buying a \$50 theme and trying to make the company fit the theme.

The big brands you mentioned have a common pattern

Look at Dave's homepage today.

It's not a generic Bootstrap hero saying:

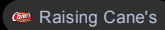
"Welcome to Dave's Hot Chicken."

It's a constantly changing branded campaign surface: huge creative, current products, aggressive food photography, bright backgrounds and direct **ORDER NOW** CTAs.

 Dave's Hot Chi...

Chipotle similarly puts ordering/location selection directly into the user journey rather than making you dig through a brochure-style corporate website. 

Cane's navigation prioritizes menu, locations, ordering and brand/community content.



Captain Jay's should follow the same philosophy.

Not copy the design.

Copy the **product thinking**.

Here's how I see the Captain Jay's homepage

I'd make it distinctly Captain Jay's.

The brand already has very strong yellow/red recognition. Don't fight it.

Imagine:

CJ LOGO	MENU	LOCATIONS	OUR STORY	BLOG	[ORDER]
WORLD FAMOUS LEMON PEPPER CHICKEN. Crispy. Juicy. Covered in Jay's flavor. [massive food image] [ORDER NOW] [VIEW MENU]					
HOW DO YOU CAPTAIN JAY'S?					
 FRIED RESTAURANT Order hot food	 FRESH MARKET Shop fresh + EBT				
CURRENT PROMOTION					
FAN FAVORITES					
[Chicken]	[Shrimp]	[Catfish]	[Wings]		
FIND YOUR CAPTAIN JAY'S [ZIP / CITY / ADDRESS]					
THE JAY'S STORY					
LATEST NEWS					

But when somebody selects **Fresh Market / EBT**, the visual content changes.

Hero becomes something like:

Fresh Seafood. Fresh Poultry. Your Neighborhood Market.

and the page emphasizes:

EBT Accepted • You Buy, We Fry® • Fresh Seafood • Fresh Chicken • Grocery Essentials

Same Captain Jay's brand.

Different customer mission.

That distinction is actually pretty unique, and I would make it a **feature of the brand**, not something you hide in a menu dropdown.

Design system

I'd start with Captain Jay's existing identity but make it significantly more polished.

Think:

Yellow: dominant brand canvas

Captain Jay's red: high-energy CTA/accent

Charcoal/near black: typography and contrast

Warm cream: calmer sections

White: clean sections

Then use massive typography, oversized product photography, controlled rounded edges and light texture.

I would avoid turning the whole site into yellow/red everywhere. Dave's works because the aggressive sections are balanced. Chipotle likewise knows when to let the food photography breathe.

The site should feel like a **restaurant brand**, not a SaaS dashboard.

That means shadow should be the component foundation, not the visual aesthetic.

Animation: use less than the templates

Maison uses GSAP, Framer Motion and Lenis smooth scrolling.

Sanit...

I would not copy all of that.

For Captain Jay's:

Motion for entrance animation: yes.

Subtle hero parallax: possibly.

Menu hover transitions: yes.

Page transitions: maybe.

Forced Lenis smooth scrolling: **no**.

Huge JS animation timelines: **no**.

Cursor effects: **absolutely no**. 😊

The food should do most of the visual work.

SEO architecture could become a major advantage

Next.js already gives you excellent tooling for dynamic metadata, OG images, sitemap and robots generation. N Nex... +2

And Next.js provides a straightforward JSON-LD approach for structured data. N Nex...

I'd produce:

```
Organization
  ↓
Captain Jay's

Restaurant
  ↓
one per location

Article
  ↓
every blog post

BreadcrumbList
  ↓
site navigation
```

Then each location could target searches such as:

fried chicken Dearborn MI

fish and chicken Detroit

EBT seafood market Detroit

halal chicken near me — only where factually applicable, obviously.

Captain Jay's Grand Rapids

And blog posts can support actual searches people make.

For example:

What Is Lemon Pepper Chicken?

Best Fish for Frying at Home

How You Buy, We Fry Works

Fresh vs Frozen Seafood

Captain Jay's Opens New [City] Location

Game Day Chicken & Wing Guide

What Can You Buy With EBT at Captain Jay's?

That's far more valuable than publishing random corporate news.

Google's own guidance still emphasizes useful, people-first content and crawlable links rather than tricks.  Google Develo...

Your ad/promotion manager should also live in Sanity

I'd make this a proper **Campaign** collection.

Marketing chooses:

Campaign

Name:

Super Bowl 50 Wing Special

Target:

☒ Fried

☐ Fresh Market

Locations:

☒ All

or selected locations

Placement:

- ☒ Homepage Hero
- ☒ Promo Strip
- ☒ Menu Banner
- ☐ Blog

Starts:

Feb 06, 2027 — 8:00 AM

Ends:

Feb 08, 2027 — 11:59 PM

Creative:

desktop.jpg

mobile.jpg

CTA:

ORDER NOW

Tracking:

superbowl 2027

Your frontend query simply asks:

TypeScript

```
campaign.startAt <= now
&& campaign.endAt >= now
&& campaign.concept.includes(currentConcept)
```

That makes the website a **marketing tool**, not just a digital brochure.

Blog editing experience

Your office/marketing users shouldn't see something resembling VS Code.

Sanity can give them:

Blog Posts

+

New Post

Title

[Captain Jay's Opens New Detroit Location]

Featured Image

[Upload]

Excerpt

[...]

Article


```
[ rich text editor ]
```

Category

```
[ News ]
```

SEO

Meta title

Meta description

OG image

Publish

```
[ Now ▼ ]
```

```
[ Schedule ]
```

And Sanity's visual editing can let them click content directly from the rendered site and modify it.  Sanity... +1

That is why I'm leaning heavily toward it for you.

Hosting

I would initially use:

Cloudflare

DNS / security

↓

Vercel

Next.js application

↓

Sanity Content Lake

CMS/content

↓

Paytronix

ordering

You can change hosting later if you need to.

Don't optimize the infrastructure prematurely.

Next.js Cache Components can now combine prerendered static content with dynamic sections, which is useful for exactly this type of content-heavy website.  Nex...

Most Captain Jay's pages should be essentially static/CDN-fast until content changes.

What I would choose if I were owning this project

I'd lock in:

Framework
Next.js 16 App Router

Language
TypeScript

UI
React 19
Tailwind CSS v4
shadcn/ui

Animation
Motion
very little GSAP, if any

CMS
Sanity

Images
Sanity Image CDN
next/image

Forms
Server Actions / Route Handlers
existing Captain Jay's systems where applicable

Ordering
Paytronix / existing ordering experience

Analytics
Google Analytics 4
Google Tag Manager

Monitoring
Sentry

Deployment
Vercel

DNS / security
Cloudflare

Source control
GitHub

Package manager
pnpm

Sanity's asset system also has a globally distributed CDN and image processing pipeline, so it's a sensible place for marketing/menu/blog photography.  Sanit...

My template decision

I would **not** buy a template yet and start modifying it.

I would first clone these two and inspect the source:

#1 SanityPress

[Open SanityPress](#)

#2 SiteEngine

[Open SiteEngine](#)

Then look at this as the closest restaurant implementation:

Grillhouse

[Open Grillhouse](#)

And this for visual restaurant ideas:

Maison

[Open Maison](#)

The current Sanity ecosystem has exactly the pieces we need: official Next.js integration, live previews, visual editing, page builders, blogs and restaurant-specific starters.

 Sanity... +1

So if you want me to make the decision today:

Framework: Next.js 

CMS: Sanity 

Base: SanityPress or SiteEngine 

UI: Tailwind + shadcn + custom Captain Jay's components 

Restaurant theme: use only for inspiration, don't let it dictate architecture 

EBT/Fried: separate indexable concept routes + remembered user preference 

Menu: structured HTML/CMS content, not menu images 

Locations: individual SEO pages for every restaurant 

Promotions/ads: first-class Sanity Campaign documents 

Blog: Sanity + visual editing + scheduled publication 

Ordering: integrate/deep-link to Paytronix rather than rebuilding commerce 

That's the architecture I'd be comfortable putting my name on as the senior engineer for this project.

And I think there's a second step we should do **before you write code**: I can research about **15–20 of the best current restaurant/fast-food website templates and actual restaurant sites, rank them specifically for Captain Jay's, and then design the exact Captain Jay's sitemap + homepage/menu/location wireframe + Sanity schema + Next.js folder structure** so you have a concrete blueprint to start building from instead of experimenting as you go.

Web search sanitypress looks really good is there any cost to use sanity it will be managed by three people in the company and using this template i can make it like chipotle or popeyes website yes please do this "**15–20 of the best current restaurant/fast-food website templates and actual restaurant sites, rank them specifically for Captain Jay's, and then design the exact Captain Jay's sitemap + homepage/menu/location wireframe + Sanity schema + Next.js folder structure** so you have a concrete blueprint to start building from instead of experimenting as you go "

Worked for 1m 47s

Yes — and after looking at the current Sanity/SanityPress ecosystem more carefully, **SanityPress is probably the direction I would choose for Captain Jay's.**

There is one important update from my previous answer: **do not start from the old SanityPress repository.** The original version was archived in May 2026. The actively maintained successor is **SanityPress with TypeGen**, which currently uses Next.js 16, Tailwind 4, Server Components, Sanity Live/Visual Editing and generated TypeScript types from your schemas and GROQ queries. It is intentionally less opinionated visually, which is actually ideal for Captain Jay's.  [GitHub](#) +1

And yes: **you absolutely can make the finished site look and behave at the level of Chipotle, Popeyes, Dave's Hot Chicken, Cane's, etc. using SanityPress.** SanityPress is the *foundation*, not the design. You would replace/build the frontend modules with a custom Captain Jay's design system while retaining its CMS, page-building, preview, SEO and content architecture. The new version intentionally doesn't force a heavily styled design system on you.  [Sanity Pr...](#)

1. Sanity cost for Captain Jay's

This is better than I expected.

You could initially pay \$0/month for Sanity

Sanity's current Free plan includes:

Free Sanity	Limit
Cost	\$0 forever
User seats	20
Datasets	2 public
Documents	10,000
API CDN requests	1,000,000/month
API requests	250,000/month
Assets	100 GB
Bandwidth	100 GB/month
Visual Editing	✓
Live Preview	✓
Presentation/page building	✓
Real-time editing	✓
Studio hosting	✓

That's vastly more than Captain Jay's needs initially. [Sanity... +1](#)

There is one catch.

Free only gives you these permissions:

Administrator and **Viewer**.

The Growth plan gives you proper:

Administrator / Editor / Developer / Contributor / Viewer

plus Comments, Tasks and Scheduled Drafts. Growth currently costs **\$15 per paid seat/month**; Viewer seats are free. [Sanity... +1](#)

For your three-person team:

Setup	Sanity cost
All 3 trusted people as Admin on Free	\$0/mo
You Admin + 2 people Viewer-only	\$0/mo
You Developer/Admin + 2 Editors on Growth	\$45/mo
Two editors + one free Viewer	\$30/mo

For development, I'd start **Free**.

For production, if the other two people need to regularly edit campaigns/blogs but shouldn't have access to developer/project settings, I'd eventually move to **Growth at \$45/month**.

You do **not need Growth merely to run the website**.

And we can even implement promotion start/end dates ourselves instead of paying for Sanity's built-in Scheduled Draft feature:

TypeScript

```
defined(startsAt) &&  
startsAt <= now() &&  
(  
  !defined(endsAt) ||  
  endsAt >= now()  
)
```

So:

```
Labor Day Special  
Starts: Sep 1 8:00 AM  
Ends: Sep 7 11:59 PM
```

can automatically appear/disappear while remaining on Free Sanity.

Built-in Scheduled Drafts become useful when you want an editor to prepare a change to an existing page and have **the publication itself** happen at a future time; that feature is included with Growth.  Sanity... +1

2. SanityPress itself costs \$0

The current SanityPress describes itself as a **free, open-source Next.js + Sanity starter**.

 Sanity Pr...

So the core stack could realistically be:

SanityPress	\$0
Sanity Free	\$0
Vercel Pro	\$20/month
Cloudflare DNS	existing/free depending setup
GitHub	\$0 or existing company account

Starting hosting ~ \$20/month

Vercel specifically states that Hobby is for personal/non-commercial use, while Pro is for businesses/professional projects. Pro currently starts at \$20/month. [▲ Vercel](#)

If Captain Jay's later goes to Sanity Growth with three editors:

Vercel Pro	\$20
Sanity Growth 3×\$15	\$45

~ \$65/month

That's extremely reasonable for a company site of this scope.

3. The SanityPress version I'd use

Use: SanityPress with TypeGen

Not the original.

Current architecture includes:

```
Next.js 16
App Router
React Server Components
TypeScript
Tailwind CSS 4
Sanity Studio
Sanity Live Content API
Visual Editing
GROQ TypeGen
next/image + Sanity CDN
Sitemap
RSS
Open Graph support
```

[GitHub +1](#)

This is nearly exactly what I would have chosen from scratch.

The current repository starts roughly like this:

```

SanityPress
├─ src/
│  ├─ app/
│  │  └─ (frontend)/
│  │     └─ layout.tsx
│  │     └─ [...slug]/
│  │     └─ blog/[slug]/
│  │     └─ blog/rss.xml/
│  │     └─ api/
│  │        └─ draft-mode/
│  │        └─ og/
│  └─ next.config.ts
├─ sanity.config.ts
├─ sanity.cli.ts
└─ package.json

```



We'd evolve that rather than fighting it.

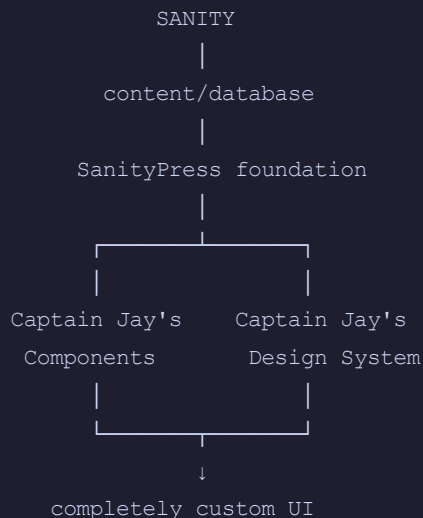
4. Can it actually look like Chipotle / Popeyes?

Yes. 100%.

This distinction is important:

SanityPress controls architecture and content management. It does not determine what Captain Jay's looks like.

Think:



Chipotle's current site, for example, is strongly campaign-driven: the hero is currently promoting Pollo Asado, immediately pushes [ORDER NOW](#), integrates store selection, and then flows into product/story/catering content. +1

Dave's currently uses aggressive brand graphics, product campaigns, huge imagery, minimal copy and repeated ordering CTAs.  Dave's Hot Chi...

Chick-fil-A takes a more modular approach: campaign hero, limited-time products, catering, menu, rewards and restaurant search are discrete reusable blocks. That's particularly relevant to how I'd architect Captain Jay's Sanity modules.  Chick-fil-A

That entire presentation layer can be built on top of SanityPress.



5. I researched 20 references — here's my Captain Jay's ranking

I separated **actual restaurant websites** from **templates/starters**, because comparing Chipotle directly with a GitHub starter doesn't make much sense.

Real restaurant websites — what Captain Jay's should borrow

Rank	Website	CJ Fit	What I would steal conceptually
1	Dave's Hot Chicken	9.8/10	Bold visual identity, food-first hero, campaigns, aggressive ordering CTA
2	Chipotle	9.7/10	Store-first ordering, campaign architecture, location handling, modular pages
3	Chick-fil-A	9.5/10	Homepage modularity, location pages, campaigns, rewards/app integration
4	Popeyes	9.3/10	Location-aware menus/pricing, fried-chicken category UX, offers
5	Sweetgreen	9.0/10	Beautiful structured menu, nutrition, real HTML content, photography

Rank	Website	CJ Fit	What I would steal conceptually
6	Raising Cane's	8.8/10	Very clear brand, minimal menu/navigation, storytelling
7	Shake Shack	8.6/10	Premium visual treatment and sophisticated location finder
8	Jollibee	8.5/10	Family-oriented brand + product pages + local location SEO
9	Wingstop	8.3/10	Massive SEO location directory architecture
10	KFC	8.1/10	Locator filters, ordering/location architecture

Dave's homepage is particularly close to where I'd take the **Fried Captain Jay's** personality. 🍗 Dave's Hot Chi... +1

Chipotle is probably the best reference for the **product architecture**. Its website connects campaigns → location → pickup/delivery → menu rather than treating the website as a digital brochure. 🌮 Chipotle +1

Chick-fil-A is probably the best reference for **location detail pages**. Individual locations include address, open status, phone, ordering options, hours, service-specific hours and amenities. 🐔 Chick-fi...

Popeyes also requires a location before exposing availability/prices, which matters if Captain Jay's menu/price availability varies by restaurant. 🍗 Pope... +1

Sweetgreen exposes menu items as actual structured content, including ingredients, nutrition and allergens rather than menu-image PDFs. 🌿 sweetgreen

Wingstop and KFC are especially worth studying for large-scale local SEO. Wingstop has crawlable country/state/location directory structures, while KFC supports locator filters such as delivery, drive-thru and catering. 🍗 Wingstop Locat... +1

6. Template/starter ranking

This is where my recommendation gets more decisive.

Rank	Starter	CJ Fit	Verdict
🏆 1	SanityPress TypeGen	9.8/10	Use this

Rank	Starter	CJ Fit	Verdict
2	Official Sanity + Next.js Clean	9.6	Best alternative
3	SiteEngine	9.4	Excellent shadcn base
4	CMS-Kit Sanity	9.0	Strong production architecture
5	Grillhouse	8.6	Best ready-made Next restaurant template
6	Maison	8.3	Great visual inspiration
7	Tasteful	8.1	Excellent restaurant CMS, wrong framework
8	Sanity Blog Theme	7.9	Useful blog/page architecture
9	The Burger	7.7	Great fast-food visual inspiration
10	Foodly	7.3	Useful menu/CMS inspiration

#1 SanityPress TypeGen — winner

It gives you the architecture without forcing somebody else's restaurant design on Captain Jay's.

It currently includes Next.js 16, Tailwind 4, TypeGen, live content/visual editing, sitemap, RSS, image optimization and modular content architecture. [GitHub](#)

This is what I would start from.

#2 Official Sanity + Next.js Clean

This one is interesting because the latest version uses **Next.js 16** and includes visual editing, live content and drag-and-drop page building. [Ver...](#)

If SanityPress somehow causes problems, this would be my fallback.

#3 SiteEngine

Really good stack:

```
Next.js
Sanity
TypeScript
Tailwind
shadcn
Zod
```

plus a 12-block page builder and Visual Editing.  Sanity...

This is very close to how I'd structure a custom corporate build.

#4 CMS-Kit

More of a serious CMS boilerplate than a restaurant template: Next.js, Sanity, real-time editing, TypeScript, Radix and configurable themes.  Ver...

Good code reference.

#5 Grillhouse

This is the strongest restaurant-specific Next.js/Sanity template I found.

It already supports:

```
menu
gallery
blog
FAQ
Sanity Studio
Next.js
restaurant content
```

The downside is Chakra UI v2 + GSAP rather than the Tailwind-based stack I'd prefer.

 Sanity...

I would **study its schemas**, not build Captain Jay's on it.

#6 Maison

Strong visual restaurant implementation with Sanity-powered menus, featured dishes, gallery and animated storytelling.  Sanity...

Beautiful, but too "fine dining" for Captain Jay's.

#7 Tasteful

This one is surprisingly complete: Sanity manages pages, menu items and blogs; it includes TypeScript, Tailwind 4, SEO, reservations and accessibility work.  Sanity...

But it's Astro.

Good reference.

Not our framework.

#8 Sanity Blog Theme

Useful specifically for your SEO/blog side because it already has Page Builder + SEO fields + blog + editor-friendly Studio. [Sanit...](#)

#9 The Burger

This was published only days ago and is actually visually relevant: bold fast-casual styling, CMS menu, product imagery and multi-location content. [Framer](#)

It's Framer, so **design reference only**.

#10 Foodly

CMS menu/categories, strong imagery, hours, maps and responsive restaurant layouts. Again, use it for visual inspiration rather than implementation. [Fra...](#)

7. Therefore my final Captain Jay's stack

I would lock this now and stop framework-shopping:



8. The sitemap I would build

This is the exact direction I recommend.

```
captainjays.net
|
├─ /
|
├─ /fried
|   └─ /fried/menu
|
├─ /fresh-market
|   └─ /fresh-market/menu
|
├─ /offers
|
├─ /locations
|   |
|   └─ /locations/michigan
|       |
|       └─ /detroit
|           └─ /[store-slug]
|
|       └─ /dearborn
|           └─ /[store-slug]
|
|       └─ /grand-rapids
|           └─ /[store-slug]
|
├─ /blog
|   └─ /category/[slug]
|       └─ /[post-slug]
|
├─ /our-story
├─ /you-buy-we-fry
├─ /rewards
├─ /app
├─ /careers
├─ /contact
├─ /feedback
|
├─ /nutrition-allergens
├─ /privacy
├─ /terms
|
├─ /sitemap.xml
└─ /robots.txt
```

9. EBT / Fried architecture — I would change one thing in your terminology

Internally, **do not model a restaurant as simply** `EBT` **or** `nonEBT`.

That's coupling two unrelated concepts.

Instead:

TypeScript

```
storeFormat:
  | "fried"
  | "fresh-market"

acceptsEBT: boolean
```

Why?

Because later Captain Jay's could theoretically have:

```
Fresh Market + EBT
Fresh Market without EBT
Fried restaurant accepting some EBT products
Hybrid store
```

You don't want to redesign your database because business rules changed.

This is what I mean by treating it like a production system rather than a website.

10. First-time customer flow

This is what I would implement.

```
Customer visits
captainjays.net
```



```
| HOW DO YOU CAPTAIN JAY'S? |
```



```
| FRIED RESTAURANT |
```

```
| Hot, crispy favorites |
```

```
| [ Explore Fried ] |
```



```
| FRESH MARKET |
```

```
| Fresh seafood + poultry |
```

```
| EBT accepted at participating |
```

```
| locations  
| [ Explore Market ]  
|
```

Selection sets:

```
cj_store_format=fried
```

or:

```
cj_store_format=fresh-market
```

in a cookie.

Then:

```
Fried  
  ↓  
/fried  
  
Fresh Market  
  ↓  
/fresh-market
```

Future visits to  can server-side redirect them to the experience they previously chose.

And the header always has:

```
Fried ▼
```

or:

```
Fresh Market ▼
```

so switching is effortless.

11. Don't make contain two hidden versions

This is important for SEO.

Don't do:

TypeScript

```
if (cookie === 'fried')  
  return <FriedHomepage />  
  
return <MarketHomepage />
```


while everything remains `/`.

Instead let Google have real URLs:

```
/fried
/fresh-market
```

while cookies make the experience feel personalized.

That gives you separate:

```
<title>
meta descriptions
OG images
copy
structured content
internal links
campaigns
menus
```

for the two concepts.

12. Fried homepage wireframe

Here's what I'd actually build.

CJ LOGO	MENU	LOCATIONS	OFFERS	OUR STORY	[ORDER]
FRIED ▼					
WORLD FAMOUS					
LEMON PEPPER CHICKEN					
Crispy. Juicy. Covered in Jay's flavor.					
[ORDER NOW]		[VIEW MENU]			
HUGE CHICKEN					
PRODUCT IMAGE					
LIMITED TIME / PROMOTION					
20 WINGS. LEMON PEPPER. GAME NIGHT.					
[ORDER NOW]					
FAVORITES					
CHICKEN	WINGS	FISH	SHRIMP	SIDES	
[photo]	[photo]	[photo]	[photo]	[photo]	



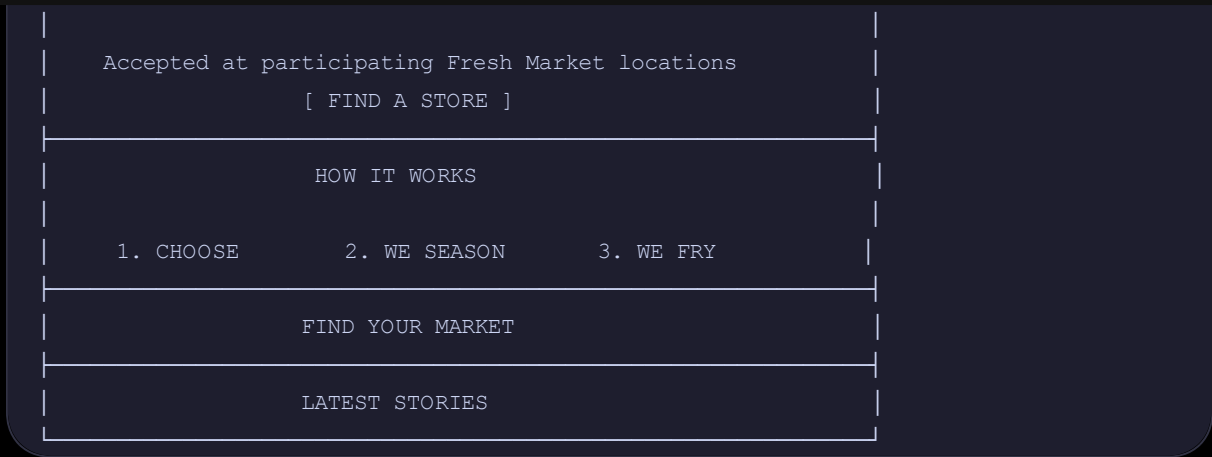
Dave's and Chick-fil-A both validate this campaign/product-first philosophy: their current homepages lead with current products/offers rather than corporate history.

 Dave's Hot Chi... +1

13. Fresh Market homepage should be meaningfully different

Not merely changing one hero photograph.





And this plays directly into Captain Jay's existing positioning: the current website already emphasizes both the Fresh Market concept and **You Buy, We Fry®**, plus seafood, poultry, produce and essentials.  Captain Jay... +1

That's an advantage competitors don't really have.

We should exploit it.

14. Menu page

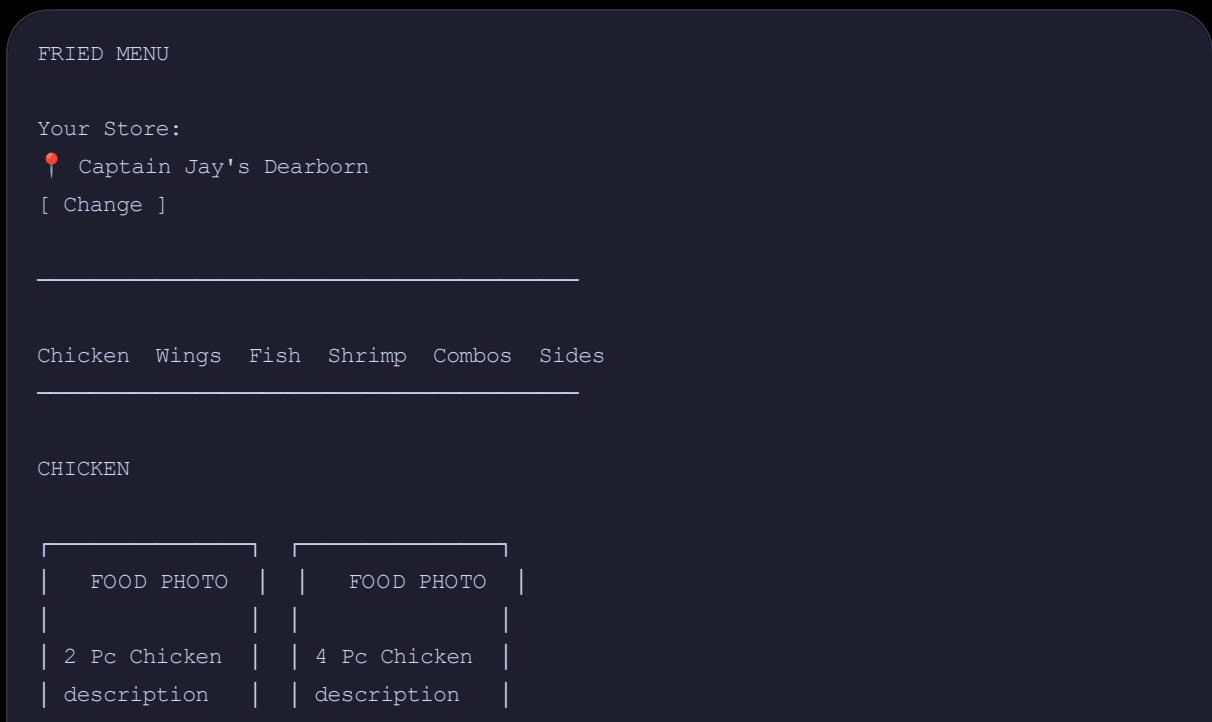
The current menu page is one of the biggest opportunities.

Right now Google's crawl of your Fried Menu essentially gets:

“Swipe Left Or Right / Click to Open Full View”

instead of meaningful menu content.  Captain Jay's ...

I would replace that completely.



calories?	calories?
[ORDER]	[ORDER]

WINGS

...

FISH

...

For menu architecture, Sweetgreen is particularly good reference material because every item is genuine HTML content with names, ingredients, calories and nutrition rather than an image menu.  sweetgr...

15. Important Paytronix decision

I would keep these separate:

Sanity

name
description
image
category
SEO content
featured item
nutrition copy
display order

Paytronix

actual ordering
actual price
store availability
modifiers
cart
payment
rewards

Don't create two competing sources of truth.

Especially for prices.

Popeyes and Chick-fil-A both use location selection because menu pricing/availability can differ by restaurant. Chick-fil-A explicitly tells users prices vary by location.  Pope... +1

Captain Jay's should behave similarly.

16. Location page — this could be massive for SEO

Every Captain Jay's should have a real page.

Example:

```
captainjays.net/  
locations/  
michigan/  
dearborn/  
telegraph-road
```

Page:

Captain Jay's Fish & Chicken
Dearborn - Telegraph Road

2657 S Telegraph Rd
Dearborn, MI 48124

OPEN NOW • Closes 10 PM

[ORDER NOW] [DIRECTIONS] [CALL]

Fried Restaurant
Fresh Market
EBT Accepted
Delivery
Pickup

HOURS

Mon	10 AM - 10 PM
Tue	10 AM - 10 PM
...	

MENU FAVORITES

Chicken | Wings | Fish | Shrimp

CURRENT OFFERS

ABOUT THIS LOCATION

Fresh local paragraph

MAP

FAQ

Does this Captain Jay's accept EBT?

Does this location offer delivery?

Can I order online?

Does this location have You Buy. We Fry?

This is very similar conceptually to Chick-fil-A's current local pages, which contain address, hours, phone, ordering methods, service options and amenities.  Chick-fi...

And Google explicitly supports `Restaurant` structured data containing address, geographic coordinates, telephone, menu URL and opening hours.  Google Develo... +1

This should be a **priority**, not a future nice-to-have.

17. Sanity content model

Here is the schema architecture I would build.

Schema	Purpose
<code>siteSettings</code>	Global branding/navigation/footer
<code>storeFormat</code>	Fried / Fresh Market
<code>location</code>	Every restaurant
<code>menu</code>	Fried/market menu sets
<code>menuCategory</code>	Wings/Fish/etc
<code>menuItem</code>	Actual products
<code>campaign</code>	Ads/offers/promos
<code>page</code>	CMS page builder
<code>blogPost</code>	SEO/editorial content
<code>blogCategory</code>	Blog organization

Schema	Purpose
<code>author</code>	Post authors
<code>announcement</code>	Site alert bars
<code>faq</code>	Reusable FAQs
<code>redirect</code>	SEO redirects
<code>globalModule</code>	Reusable page sections

18. `location` is your most important schema

Something like:

TypeScript

```
location {
  name
  slug

  storeNumber

  address {
    street
    city
    state
    zip
  }

  geo {
    lat
    lng
  }

  phone

  storeFormat
  acceptsEBT

  hours[]
  holidayHours[]

  services[] {
    pickup
    delivery
    catering
    freshMarket
    youBuyWeFry
  }
}
```

```
}

paytronixStoreId
orderId

menuProfile

images[]

seo {
  title
  description
  image
}
}
```

19. Menus

I would use:

```
menu
↓
menuCategory[]
↓
menuItem[]
```

Example:

TypeScript

```
menuItem {
  name: "20 Piece Wing Dings"

  slug: "20-piece-wing-dings"

  description:
    "Crispy wing dings tossed in your favorite seasoning."

  image

  category -> Wings

  formats: ["fried"]

  featured: true

  nutrition {
    calories
    protein
    allergens
  }
}
```



```
paytronixProductId

seo {
  ...
}
```

This also means one menu item can be used in:

```
homepage
menu
promotion
blog
location
related products
```

without duplicating it.

20. Campaign/advertisement management

This is the thing I particularly want to build well for you.

TypeScript

```
campaign {
  title

  campaignType:
    "hero"
    "offer"
    "announcement"
    "menu-promo"
    "app-promo"

  audience:
    "fried"
    "fresh-market"
    "both"

  locations[]

  headline
  subheadline

  desktopImage
  mobileImage

  cta {
    label
    url
  }
}
```

```
    startAt
    endAt

    priority

    trackingId

    enabled
  }
```

Now marketing could create:

 SUPER BOWL WEEKEND

Audience:

☒ Fried

☐ Fresh Market

Locations:

☒ All Fried Stores

Homepage Hero:

☒

Start:

Feb 05 8:00 AM

End:

Feb 09 11:59 PM

CTA:

ORDER NOW

No deployment.

No contacting you.

No code change.

21. Blog schema

TypeScript

```
blogPost {
  title
  slug

  excerpt

  featuredImage
}
```

```

author

categories[]

publishedAt

content[]

relatedLocations[]
relatedMenuItems[]

seo {
  metaTitle
  metaDescription
  ogImage
  canonical
  noIndex
}
}

```

And output `BlogPosting` structured data. Google specifically documents `Article`, `NewsArticle` and `BlogPosting` structured data for article pages.  Google Develo...

22. Page builder

This is where SanityPress becomes really valuable.

Instead of hardcoding:

```
Homepage = hero + products + story + blog
```

we define modules:

```

Hero
Campaign Hero
Split Hero
Menu Categories
Featured Products
Location Finder
Promotion
You Buy We Fry
Image + Text
Video
Story
App / Rewards
Blog Posts
FAQ
CTA
Stats
Gallery
Rich Text

```

Then Sanity:

Homepage

1. Campaign Hero
2. Featured Categories
3. Promotion
4. Location Finder
5. You Buy We Fry
6. App CTA
7. Latest Blog Posts

[+ Add Module]

Marketing can rearrange things themselves.

That's the right abstraction.

23. My Captain Jay's Next.js folder structure

I'd evolve SanityPress into this:

```
src/
|
├─ app/
|   |
|   └─ (frontend) /
|       |
|       └─ layout.tsx
|       └─ page.tsx
|       |
|       └─ fried/
|           └─ page.tsx
|               └─ menu/
|                   └─ page.tsx
|       |
|       └─ fresh-market/
|           └─ page.tsx
|               └─ menu/
|                   └─ page.tsx
|       |
|       └─ locations/
|           └─ page.tsx
|               └─ [state]/
|                   └─ [city]/
|                       └─ [store]/
|                           └─ page.tsx
|       |
|       └─ offers/
|           └─ page.tsx
```

```
|
|
|   |
|   └─ blog/
|       |
|       └─ page.tsx
|           |
|           └─ [slug]/
|               |
|               └─ page.tsx
|
|   |
|   └─ our-story/
|       |
|       └─ rewards/
|           |
|           └─ careers/
|               |
|               └─ contact/
|                   |
|                   └─ sitemap.ts
|                       |
|                       └─ robots.ts
|
|   └─ admin/
|       |
|       └─ [[...tool]]/
|
|   └─ components/
|       |
|       └─ brand/
|           |
|           └─ logo.tsx
|               |
|               └─ header.tsx
|                   |
|                   └─ footer.tsx
|
|       |
|       └─ home/
|           |
|           └─ menu/
|               |
|               └─ menu-grid.tsx
|                   |
|                   └─ menu-card.tsx
|                       |
|                       └─ category-nav.tsx
|                           |
|                           └─ store-menu-header.tsx
|
|       |
|       └─ locations/
|           |
|           └─ location-card.tsx
|               |
|               └─ location-search.tsx
|                   |
|                   └─ store-hours.tsx
|
|       |
|       └─ campaigns/
|           |
|           └─ blog/
|               |
|               └─ modules/
|                   |
|                   └─ hero/
|                       |
|                       └─ campaign-hero/
|                           |
|                           └─ featured-products/
|                               |
|                               └─ location-finder/
|                                   |
|                                   └─ image-text/
|                                       |
|                                       └─ faq/
|                                           |
|                                           └─ ...
|
|   └─ sanity/
|       |
|       └─ schemaTypes/
|           |
|           └─ documents/
```

```

| | | | └─ location.ts
| | | | └─ menu.ts
| | | | └─ menuItem.ts
| | | | └─ campaign.ts
| | | | └─ blogPost.ts
| | | | └─ page.ts
| | | |
| | | └─
| | └─ objects/
| |   └─ seo.ts
| |   └─ hours.ts
| |   └─ address.ts
| |   └─ modules/
| |
| └─ queries/
|   └─ locations.ts
|   └─ menu.ts
|   └─ campaigns.ts
|   └─ blog.ts
|
| └─ lib/
|
| └─ lib/
|   └─ analytics/
|   └─ paytronix/
|   └─ seo/
|   └─ stores/
|   └─ cookies/
|
| └─ styles/

```

That is clean enough that another good engineer should be able to open the repo two years from now and understand what's happening.

24. Performance philosophy

For Captain Jay's I want **very little browser JavaScript**.

Default:

Server Component

Only use client components for:

store selector
location search/map
mobile nav
carousel when necessary
concept selector
interactive menus
analytics integrations
small animations

Everything else:

```
HTML generated on server
  ↓
Vercel CDN
  ↓
customer
```

Images:

```
Sanity CDN
+
next/image
+
AVIF/WebP
+
responsive sizes
```

Fonts:

```
next/font
self-hosted
```

Animation:

```
CSS
Motion
```

instead of loading a giant GSAP animation architecture everywhere.

This is how we get the **smooth** feeling you're asking for without creating a slow website disguised as a fancy website.

25. SEO strategy becomes much stronger

Captain Jay's currently says it has more than twenty locations around Metro Detroit and Grand Rapids.  Captain Jay's ...

Imagine all of those producing indexable pages:

```
Captain Jay's Detroit
Captain Jay's Dearborn
Captain Jay's Grand Rapids
Captain Jay's Taylor
Captain Jay's Southfield
...
```

Then articles:

```
Best Fish for Frying
```

```
What Is Lemon Pepper Chicken?
```

```
What Does You Buy, We Fry Mean?
```

```
Does Captain Jay's Accept EBT?
```

```
Where Can I Buy Fresh Seafood in Detroit?
```

```
Captain Jay's Opens [New Location]
```

```
How to Choose Fresh Catfish
```

```
Game Day Chicken & Wing Guide
```

And internally link:

```
blog
↓
menu items
↓
locations
↓
order
```

That's a real SEO content architecture.

Not "put keywords in a page and hope."

26. One more thing I'd borrow from Jollibee

Jollibee creates indexable individual **product pages**, not only one generic menu page. Its product pages expose product description, ordering information, nutrition and allergen links.  Jollibee Foods ... +1

We don't need this in V1, but eventually:

```
/menu/chicken-sandwich
/menu/wing-dings
/menu/catfish
/menu/large-shrimp
```

could be useful.

Especially for branded/signature products.

27. Recommended build order

Don't try to build everything simultaneously.

Phase	Build
0	Content inventory + Captain Jay's design system
1	SanityPress TypeGen + Vercel + Sanity
2	CMS schemas + page builder
3	Header/footer + Fried/Fresh Market routing
4	Fried and Fresh Market homepages
5	Structured menu system
6	Locations + individual store pages
7	Campaign/advertisement manager
8	Blog + SEO
9	Paytronix links/integration
10	Analytics, performance, accessibility, redirects
11	Content migration + QA + launch

I would **not even begin by building the homepage.**

First:

```
Design tokens
↓
Content model
↓
Routes
↓
Components
↓
Pages
```

That prevents you from ending up with another AI-generated pile of components that happens to look pretty.

28. Final decision

If I were the senior engineer responsible for Captain Jay's, I'd sign off on:

SanityPress TypeGen + Next.js 16 + TypeScript + Tailwind 4 + Sanity + Vercel + Cloudflare + Paytronix

And I would use the references this way:

```
Dave's Hot Chicken
→ Captain Jay's visual boldness

Chipotle
→ ordering / location UX

Chick-fil-A
→ modular homepage + location pages

Popeyes
→ location-aware menu

Sweetgreen
→ structured food/menu content

Raising Cane's
→ brand simplicity

SanityPress
→ actual engineering foundation
```

Don't buy Grillhouse. Don't move to Framer. Don't move to Webflow. Don't start from scratch.

Start from **the current SanityPress TypeGen**, rip out its generic presentation, and build a proper Captain Jay's component library over its CMS infrastructure.

The combination of **Fried + Fresh Market + You Buy, We Fry® + EBT + strong Detroit identity** actually gives Captain Jay's more interesting material to work with than a lot of the chains we're studying. The new website shouldn't look like a smaller Popeyes. It should look like **Captain Jay's became a national-quality brand.**  Captain Jay's ... +1

The next thing I'd do before writing production code is define the **visual design system**: exact yellow/red/cream/black palette, typography, button system, cards, nav, hero treatment, photography rules, spacing scale, animation rules, desktop/mobile header, and then a **high-fidelity homepage design modeled on the strongest ideas from Dave's + Chipotle + Chick-fil-A while remaining unmistakably Captain Jay's**. That will determine whether this becomes merely technically good or actually looks like the national-chain-quality site you're aiming for.

